ProducerConsumerExample (HIGH):->

```
SharedBuffer.java:->
```

```java
package com.example.demo;

public class SharedBuffer {
    private int data;
    private boolean hasData=false;

    //producer
    public synchronized void produce(int value)
    {
        while(hasData)
        {
            try
            {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        data=value;
        System.out.println(Thread.currentThread().getName()+" produced:
"+data);
        hasData=true;
        notify();
    }

    //consume
    public synchronized void consume()
    {
        while(!hasData)
        {
            try
            {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        System.out.println(Thread.currentThread().getName()+" consumed:
"+data);
        hasData=false;
        notify();
    }
}
```

MainTest.java:->

```java
package com.example.demo;

public class MainTest {
    public static void main(String[] args) {
        //producer consumer example
        SharedBuffer sharedBuffer = new SharedBuffer();
        //producer thread
        Thread producerThread = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sharedBuffer.produce(i);
                try {
                    Thread.sleep(500L);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }, "Producer Thread");


        //consumer thread

        Thread consumerThread = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sharedBuffer.consume();
                try {
                    Thread.sleep(800L);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }


        }, "Consumer Thread");
        producerThread.start();
        consumerThread.start();
    }
}
```

SharedBufferWithLock.java:->

```
package com.example.demo;

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class SharedBufferWithLock {
    private int data;
    private boolean hasData=false;
    private final Lock lock=new ReentrantLock();
    private final Condition condition=lock.newCondition();

    //producer

    public void produce(int value) {
        lock.lock();
        try {
            while (hasData) {
                condition.await();
            }
            data = value;
            System.out.println(Thread.currentThread().getName() + " produced:
" + data);
            hasData = true;
            condition.signal();

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();

        } finally {
            lock.unlock();
        }
    }
//consumer
    public void consume()
    {
        lock.lock();
        try
        {
            while(!hasData)
            {
                condition.await();
            }
            System.out.println(Thread.currentThread().getName()+" consumed:
"+data);
            hasData=false;
            condition.signal();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }finally {
            lock.unlock();
        }    }
}
```

MainTest1.java:->

```java
package com.example.demo;

public class MainTest1 {
    //producer consumer example with lock
    public static void main(String[] args) {


        SharedBufferWithLock sharedBufferWithLock = new
SharedBufferWithLock();
        Thread producer = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sharedBufferWithLock.produce(i);
                try {
                    Thread.sleep(500L);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }, "Producer Thread");

        //consumer
        Thread consumer = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sharedBufferWithLock.consume();
                try {
                    Thread.sleep(800L);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }, "Consumer Thread");
        producer.start();
        consumer.start();

    }
}
```